

三模式可重构 MRAM 存内计算 SNN 架构完整方案

方案名称：Unified Granularity-Reconfigurable MRAM-IMC SNN Architecture，简称 **UGR-MRAM-SNN**。

本方案在原“Temporal-block Pipelined Layer-major SNN”基础上扩展，使同一套硬件可重构支持：逐时间步优先、全窗口层优先和时间块混合三种执行模式。

为避免“层优先”和“时间步优先”的中文歧义，全文以外层循环命名，并同时给出循环顺序。

与需求中的表述对应如下：

“全部层优先”：固定一个 timestep，先让它完成全部网络层 → Time-major

“全部时间步优先”：固定一个 layer，先让它完成全部 timestep → Layer-major

“混合策略”：固定一个 temporal block，块内逐层完成全部 timestep → Hybrid

0. 方案摘要

本方案面向连续 EEG/LFP 癫痫检测与闭环刺激，也适用于其他流式 SNN 推理任务。其核心不是设计三套独立数据通路，而是建立一个统一的、由时间块长度控制的执行模型：

$$B \in [1, T]$$

其中：

- T 为一次推理或检测窗口包含的 SNN 时间步数；
- B 为调度器一次交给某一网络层连续处理的时间步数；
- $K = \lceil T/B \rceil$ 为窗口内时间块数量。

三种模式统一为：

模式	模式编码	块长	外层循环	主要目标
逐时间步优先，Time-major	TM=00	$B = 1$	时间步在外、层在内	最低反馈延迟
时间块混合，Hybrid block	HYB=01	$1 < B < T$	块在外、块内层在外	延迟、吞吐和能耗折中
全窗口层优先，Layer-major	LM=10	$B = T$	层在外、该层处理全部时间步	最大连续复用和批处理效率

统一循环可表示为：

```
for block in partition(time_window, block_len=B):
    for layer in network_layers:
```

```
for timestep in block:
    run_mram_imc_and_neuron_update(layer, timestep)
```

因此：

$B = 1$	→ 每个时间块只有 1 个 timestep	→ 逐时间步优先
$1 < B < T$	→ 每个时间块内采用层优先	→ 时间块混合
$B = T$	→ 整个窗口只有 1 个时间块	→ 全窗口层优先

硬件中共享以下资源：

同一份常驻 MRAM 权重
同一组 MRAM-IMC 计算簇与 ADC/累加外围
同一组 LIF 神经元状态存储
同一套输入、层间和回放缓冲
同一套风险融合及闭环控制器

模式切换时仅重构：

调度顺序
时间块长度
互连路由
缓冲区分区与读写指针
输出聚合节拍
功耗门控策略

正常模式切换不改写 MRAM 权重，从而避免写延迟、写能耗和器件耐久度开销。

1. 设计目标与边界

1.1 设计目标

本架构需要同时满足：

1. 同一芯片支持三种执行模式，并允许运行时切换；
2. 三种模式共享权重和计算阵列，不复制整套网络；
3. 在数学依赖允许时，三种模式对相同输入产生逐时间戳一致的 SNN 结果；
4. 支持连续输入、分块流水、风险输出和闭环刺激；
5. 模式切换、刺激后 flush、状态 reset 均不能产生旧数据误触发；
6. 支持 MRAM 差分权重、位切片、多阵列拼接和量化 ADC；
7. 支持可观测、可验证、可综合的控制接口和性能计数器。

1.2 目标网络模型

第一版以无跨层循环的前馈多层 SNN 为目标：

$$S_{l-1}(t) \rightarrow W_l \rightarrow U_l(t) \rightarrow S_l(t)$$

每层允许存在时间递归状态：

$$U_l(t) \leftarrow f(U_l(t-1), I_l(t), S_l(t-1))$$

但默认不允许会破坏调度可交换性的任意跨层反馈，例如：

$$S_3(t) \rightarrow S_1(t+1)$$

若网络包含跨层循环、侧向循环、动态归一化或其他全局时间依赖，编译器必须分析依赖关系，并执行以下一种策略：

强制使用 TM 模式
限制最大块长 B
插入同步屏障
将循环子图固定映射为不可重排计算域

1.3 数值等价条件

三种模式本质上改变合法计算节点的执行顺序，而不改变依赖图。对于标准前馈 SNN，第 l 层第 t 步节点依赖：

$$N(l, t) \leftarrow \{N(l-1, t), N(l, t-1)\}$$

只要调度器同时满足：

同层时间依赖： $N(l, t-1)$ 必须先完成
同时间跨层依赖： $N(l-1, t)$ 必须先完成

则 TM、HYB 和 LM 都是该有向无环展开图的合法拓扑序。固定量化、舍入、饱和及随机数种子后，三种模式应产生相同的逐时间戳 spike、膜电位和 logits；差别主要体现在结果到达的物理时间、缓冲需求、资源利用率和能耗。

2. 三种执行模式

设网络包含 L 层，检测窗口包含 T 个时间步。

2.1 模式 TM：逐时间步优先

循环顺序：

```
for t in range(T):
    for l in range(L):
```

```
run_layer(l, t)
emit_risk_if_enabled(t)
```

执行序列：

```
t1: W1 → W2 → W3 → readout
t2: W1 → W2 → W3 → readout
t3: W1 → W2 → W3 → readout
...
```

特点：

最早可在一个 timestep 完成后产生风险更新
 激活中间结果只需保存很短时间
 适合突发高风险、闭环快速确认和低延迟控制
 块级固定开销不能被多个 timestep 摊薄
 若计算阵列折叠复用，层选择和外围上下文切换最频繁

在采用每层独立 MRAM stage 的空间映射时，TM 可形成逐时间步波前流水：

```
slot 0: Stage1(t1)
slot 1: Stage1(t2) | Stage2(t1)
slot 2: Stage1(t3) | Stage2(t2) | Stage3(t1)
slot 3: Stage1(t4) | Stage2(t3) | Stage3(t2) | Readout(t1)
```

2.2 模式 HYB：时间块混合

将输入划分为 K 个时间块：

$$\mathcal{B}_k = \{t_k, t_k + 1, \dots, t_k + B_k - 1\}$$

其中最后一个块允许：

$$B_k < B$$

块内执行 layer-major：

```
for block in blocks:
    for l in range(L):
        for t in block:
            run_layer(l, t)
        emit_block_risk(block)
```

采用每层独立 MRAM stage 时，不同 block 可在层间流水：

```

slot 0: Stage1(B1)
slot 1: Stage1(B2) | Stage2(B1)
slot 2: Stage1(B3) | Stage2(B2) | Stage3(B1)
slot 3: Stage1(B4) | Stage2(B3) | Stage3(B2) | Readout(B1)

```

特点：

每个阵列对同一层连续服务 B 个 timestep
 块级配置、寻址、DMA 和 ADC 启停开销可被 B 摊薄
 每 B 个 timestep 至少产生一次可用于闭环的块级风险
 比 TM 需要更大的层间激活缓冲
 比 LM 更早产生可用输出

推荐运行时可选：

$$B \in \{2, 4, 8, 16\}$$

第一版优先实现：

$$B = 4$$

2.3 模式 LM：全窗口层优先

循环顺序：

```

for l in range(L):
    for t in range(T):
        run_layer(l, t)
emit_window_risk()

```

执行序列：

```

W1(t1...tT)
  ↓ 保存 S1(t1...tT)
W2(t1...tT)
  ↓ 保存 S2(t1...tT)
W3(t1...tT)
  ↓
Readout

```

特点：

同一层阵列保持最长连续工作区间
配置、寻址和外围上下文切换开销最低
适合离线回放、低风险低功耗监测、大窗口统计和吞吐优先任务
必须保存一个窗口的层间 spike 序列
首个窗口结果的反馈延迟最高

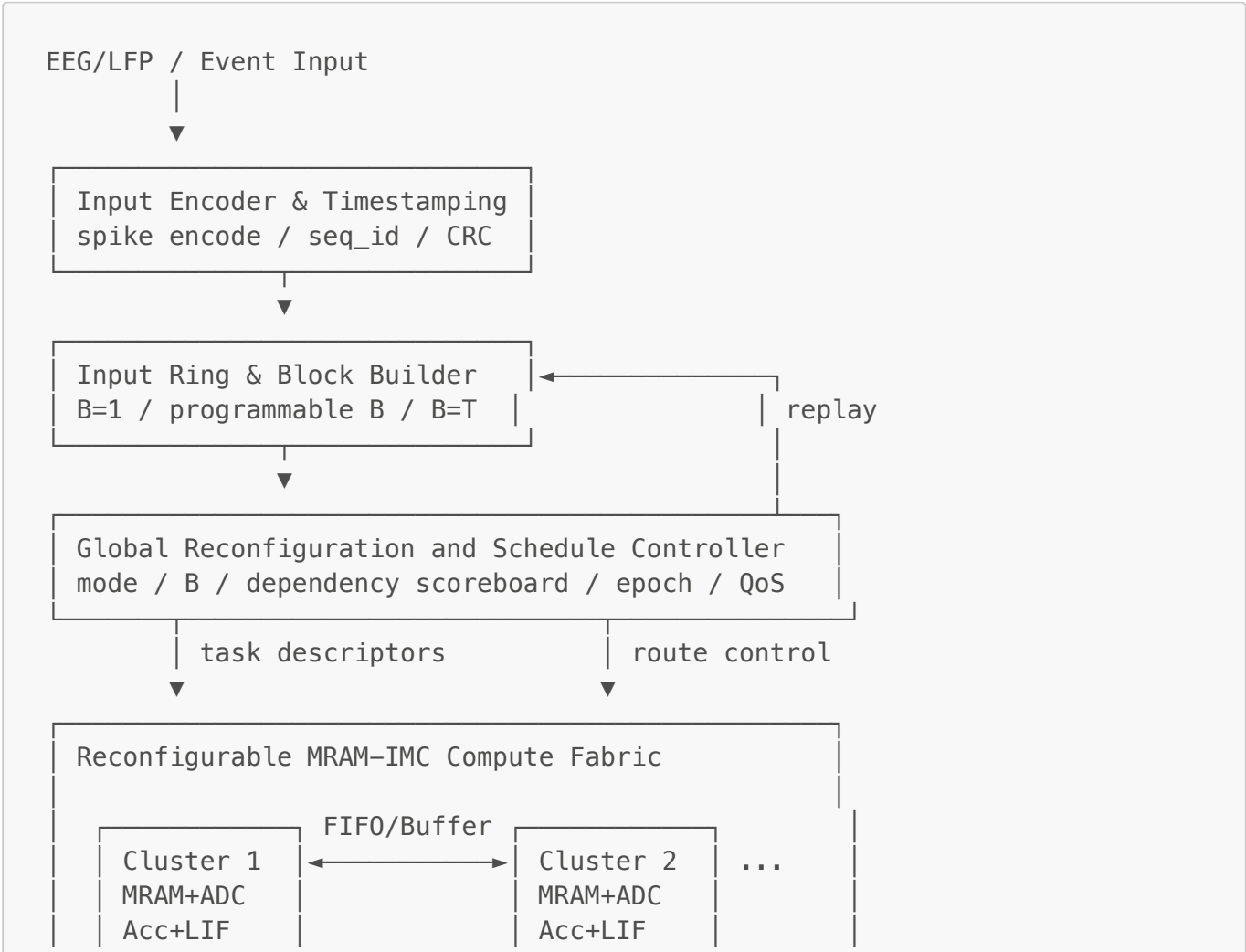
严格 LM 模式下，调度器设置层间全窗口屏障：只有 Layer l 的全部 T 个时间步完成后，Layer $l + 1$ 才开始工作。若允许层间提前消费，则已经退化为流水化 HYB，不再称为严格 LM。

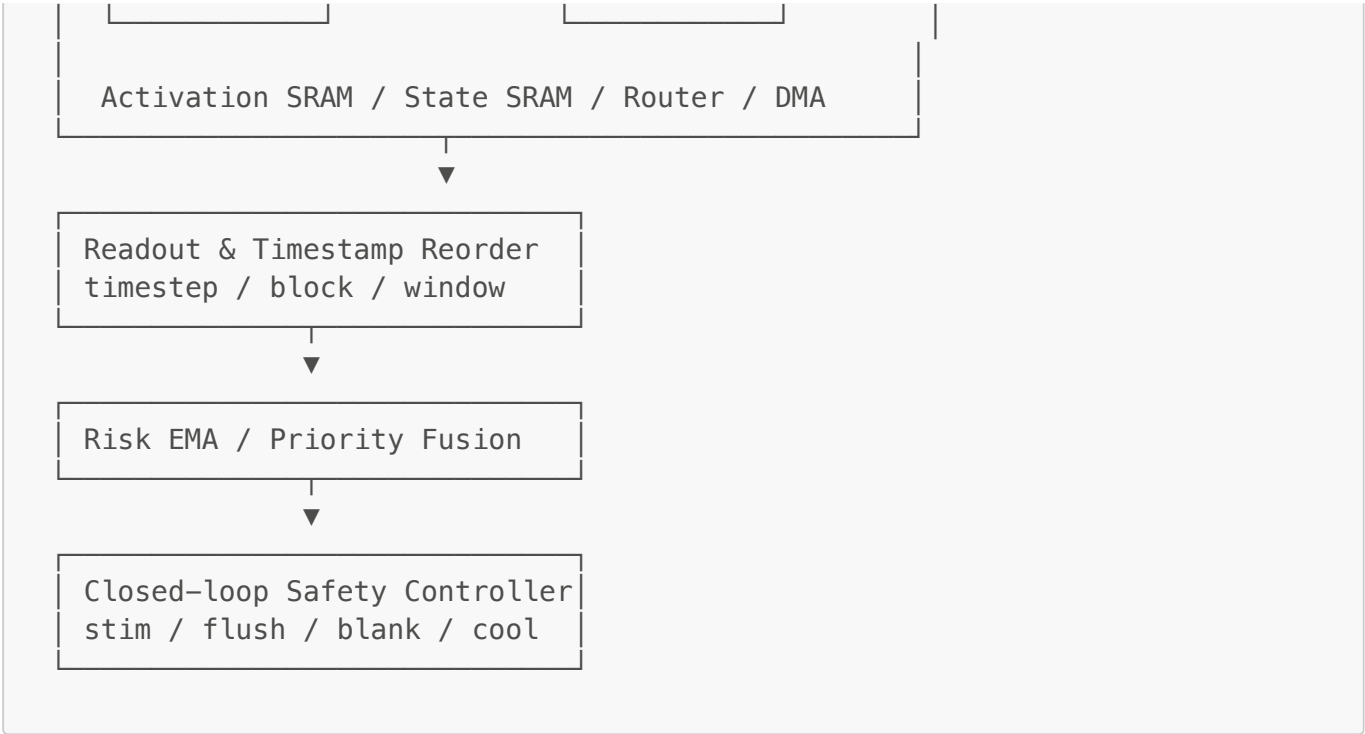
2.4 三种模式的统一关系

TM = HYB(B=1)
HYB = HYB(1<B<T)
LM = HYB(B=T, 且启用全窗口层间屏障)

因此硬件只需要一套可参数化调度器，不需要维护三份独立控制逻辑。

3. 顶层架构





3.1 主要硬件模块

模块	主要职责
Input Encoder	将连续信号编码为 spike，附加时间戳和序列号
Input Ring	保存尚未提交及可能需要回放的输入时间步
Block Builder	按模式生成长度为 B 的时间块
Global Reconfiguration Controller, GRC	接收模式请求，执行 drain、checkpoint、flush 和配置提交
Schedule Engine	生成 $(layer, block, tile, timestep)$ 任务，检查依赖和资源可用性
Layer Descriptor Table	保存每层维度、权重地址、量化、阈值、延迟和映射信息
MRAM-IMC Cluster	完成向量-矩阵乘、位切片累加、ADC 量化和校准
Neuron Engine	完成 LIF/ALIF 状态更新、阈值比较、reset 和 refractory 处理
State SRAM	保存每层膜电位、突触电流、适应状态及检查点
Activation SRAM/FIFO	保存时间块或全窗口的层间 spike
Reconfigurable Router	支持 stage-to-stage、buffer-to-stage、回放和旁路路由
Readout/Reorder	按逻辑时间戳重排输出，隔离物理执行顺序差异
Risk Fusion	产生 $risk_raw$ 、 $risk_ema$ 和置信度
Safety Controller	负责刺激触发、epoch flush、blanking、cooldown 和看门狗

4. MRAM-IMC 计算簇设计

4.1 权重常驻与层映射

推理权重长期驻留在 MRAM 中：

```
Layer 1 → MRAM bank group 1
Layer 2 → MRAM bank group 2
...
Layer L → MRAM bank group L
```

也可以在面积受限版本中采用折叠映射：

```
多个 layer 的权重驻留在不同 MRAM bank
共享 ADC、数字累加器和 Neuron Engine
调度器通过 bank_select 切换当前 layer
```

两种物理实现均支持三种模式：

映射方式	优点	代价	适用版本
每层独立 stage	支持真实 block 流水，吞吐高，控制直观	面积和 ADC 数量较大	高性能版，推荐原型
折叠共享 cluster	面积小，外围复用率高	无法让所有层同时工作，切换开销更明显	低功耗小面积版
混合映射	热点层独立、其他层折叠	编译和调度更复杂	后续优化版

原附件中的 block 间 temporal pipeline 需要至少两个可并行工作的物理 stage 或 tile 分区。若所有层只共享一个不可并行的计算簇，则仍可执行 HYB，但不能获得层间 block 重叠带来的吞吐收益。

4.2 当前 16-Macro、64 通道并行与卷积核折叠映射

当前物理实现采用：

```
16 个 Macro
每个 Macro 包含 1 个 Cluster
每个 Cluster 包含 4 个卷积 MAC lane
物理并行卷积输出通道数 = 16 × 4 = 64
```

在首组 64 个输出通道之外，其余输出卷积核按额外的 64 通道组进行折叠，不增加新的物理 Cluster，而是分时复用同一组 64 路卷积 MAC。为避免“总组数”和“额外组数”混淆，本文定义：

$$N_{add} = n$$

表示首组之外还有任意可配置的 n 组折叠卷积核，总 kernel fold 数为：

$$F = 1 + N_{add} = n + 1$$

当每组均为完整的 64 个输出通道时：

$$C_{out} = 64F = 64(n + 1)$$

因此当前结构的“64”只表示物理并行宽度，逻辑输出通道数由 Fold 总数决定。配置关系为：

```
物理并行输出通道数 = 64
额外折叠组数 Nadd = n
kernel fold 总数 F = n + 1
整组映射时逻辑输出通道数 Cout = 64(n + 1)
```

更一般地，当最后一组不满 64 个通道时：

$$F = \left\lceil \frac{C_{out}}{64} \right\rceil$$

最后一个 Fold 由 `valid_channels_last_fold` 或 lane-valid mask 屏蔽超出 C_{out} 的无效 lane。

定义：

```
f: kernel fold 编号,  $f \in [0, F-1]$ 
m: Macro/Cluster 编号,  $m \in [0, 15]$ 
q: Cluster 内卷积 MAC lane 编号,  $q \in [0, 3]$ 
```

逻辑输出通道映射为：

$$c_{out} = 64f + 4m + q$$

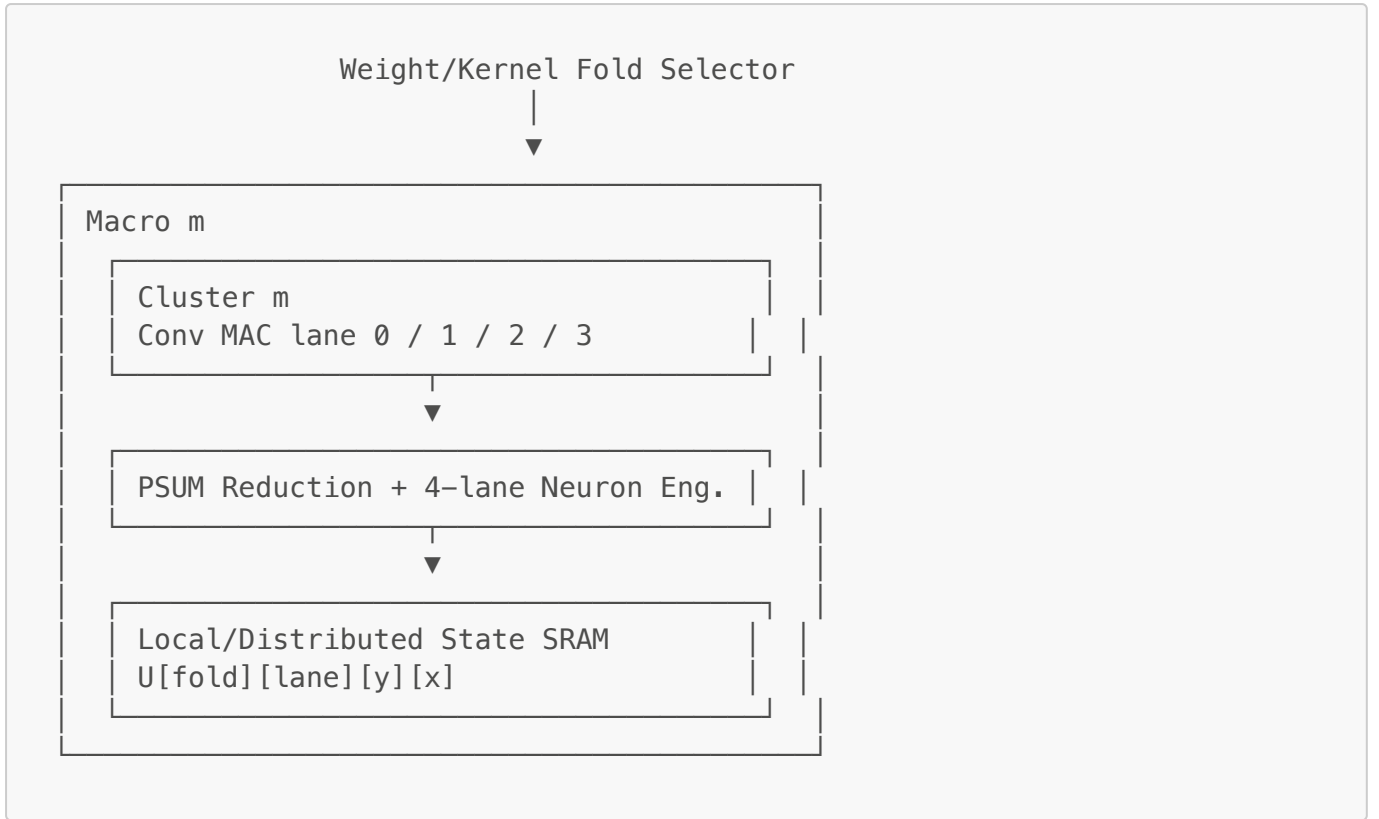
第 f 个 Fold 的通用映射为：

```
Fold f:
Macro 0 / Cluster 0 → output channel 64f + 0 ... 64f + 3
Macro 1 / Cluster 1 → output channel 64f + 4 ... 64f + 7
...
Macro15 / Cluster15 → output channel 64f + 60 ... 64f + 63
```

其中 $f = 0, 1, \dots, F - 1$ 。从物理资源看，每个 Cluster 始终只有 4 个卷积 MAC lane；全部 F 个 Fold 通过切换权重地址、输出通道基址、膜电位状态地址和 metadata 分时复用这 4 个 lane，不复制 MAC 和 LIF 数据通路。

当前折叠属于**输出卷积核/输出通道折叠**：不同 Fold 计算不同的输出通道，因此首组及额外 n 组输出均拥有互相独立的逻辑膜电位。若后续还存在输入通道、卷积核元素或权重位切片折叠，则这些子 fold 只是同一输出神经元的 partial sum，必须在完整累加后才能执行一次 LIF 更新，不能对每个 partial sum 重复施加泄漏。

推荐每个 Macro 内的数据通路为：



4.3 Spike 驱动向量-矩阵乘

第 l 层第 t 步：

$$I_l(t) = W_l S_{l-1}(t)$$

其中输入 spike 为二值时：

S=1 → 激活对应 wordline
S=0 → wordline 保持关闭

列电流近似为：

$$I_j = \sum_i V_i G_{ij}$$

经 ADC、数字校准和位切片累加后得到量化突触电流：

$$\hat{I}_l(t) = Q_{acc} \left(\sum_{p=0}^{P-1} 2^p Q_{ADC}(I_{l,p}(t)) \right)$$

4.4 有符号权重

推荐使用差分映射：

$$W_l = W_l^+ - W_l^-$$

$$I_l = \text{ADC}(I_l^+) - \text{ADC}(I_l^-)$$

可选实现：

双列差分：同一次访问读取正、负列
 双阵列差分：W+ 和 W- 分开映射
 偏置编码：减少阵列但增加数字校正

第一版推荐双列或双阵列差分，优先保证符号计算可解释性。

4.5 位宽和位切片

推荐第一版：

```
weight_bits      = 6
cell_effective_bits = 2
weight_slices    = 3
ADC_bits         = 6
accumulator_bits = 16
membrane_bits    = 16
spike_bits       = 1
```

需要比较：

```
4-bit weight + 5/6-bit ADC
6-bit weight + 6-bit ADC
8-bit weight + 8-bit ADC
```

4.6 多 tile 拼接

当矩阵尺寸超过单阵列容量时：

行方向切分 → 多 tile partial sum 数字累加
 列方向切分 → 多 tile 并行产生不同输出通道
 位方向切分 → 按 bit significance 移位累加

Layer Descriptor 中保存：

```
row_tile_count
col_tile_count
bit_slice_count
weight_bank_base
kernel_fold_count
kernel_fold_weight_base[]
output_channel_base[]
```

```
ADC_gain/offset profile
accumulation_shift
```

4.7 MRAM 非理想性处理

硬件和训练流程共同处理：

```
器件间电导偏差
读噪声
参考电流漂移
ADC offset/gain mismatch
stuck-at cell
IR drop
温度变化
```

建议加入：

```
上电和周期性 ADC 校准
阵列级 gain/offset LUT
坏点感知权重映射
关键权重 ECC 或冗余列
QAT 中注入测得的器件噪声模型
运行时饱和与异常列计数器
```

由于推理期间权重不随模式切换而重写，三模式重构不会显著消耗 MRAM 写耐久度。

5. 神经元状态与时间连续性

5.1 LIF 状态更新

对第 l 层：

$$I_l(t) = W_l S_{l-1}(t)$$

$$U_l(t) = \lambda_l U_l(t-1) + I_l(t) + b_l - V_{reset,l} S_l(t-1)$$

$$S_l(t) = \Theta(U_l(t) - V_{th,l})$$

若包含突触电流状态：

$$I_{syn,l}(t) = \alpha_l I_{syn,l}(t-1) + I_l(t)$$

则 State SRAM 至少保存：

```
U_l
I_syn,l
refractory_l
```

```
adaptive_threshold_l (若使用 ALIF)
last_committed_t_l
```

5.2 卷积核折叠下的分布式膜电位组织

对于卷积 SNN，第 l 层每个输出通道、每个输出空间位置具有独立膜电位：

$$U_l[c, y, x]$$

在当前 **16 Macro × 1 Cluster × 4 MAC lane** 结构中，一个 Cluster 的 4 个物理 MAC/LIF lane 被全部 $F = n + 1$ 个 kernel fold 分时复用，每个 Fold 映射 4 个逻辑输出通道。因此第 m 个 Cluster 的膜电位地址空间为：

$$U_m[f, q, y, x]$$

其中：

```
f ∈ [0, F-1]: Fold f 对应通道 64f ... 64f+63
q ∈ [0, 3]: Cluster 内 4 个卷积 MAC/Neuron lane
(y, x): 输出特征图空间位置
```

每个 Cluster 逻辑上负责：

$$N_{U, cluster} = 4FH_{out}W_{out}$$

个膜电位状态。对于额外 n 组折叠卷积核， $F = n + 1$ ，因此：

$$N_{U, cluster} = 4(n + 1)H_{out}W_{out}$$

全芯片 16 个 Cluster 合计：

$$N_{U, total} = 16 \times 4FH_{out}W_{out} = 64FH_{out}W_{out}$$

即整组映射时：

$$N_{U, total} = 64(n + 1)H_{out}W_{out}$$

这里需要区分：

```
物理 LIF/Neuron lane: 每个 Cluster 4 路，共 64 路
逻辑膜电位状态: 每个 Cluster 为 4F 个逻辑输出通道保存完整空间状态
```

全部 F 个 Fold 可复用同一组 4 路物理 LIF，但任意两个不同 Fold 绝不能复用同一组逻辑膜电位地址。Fold 切换至少同时切换：

```
weight_base
output_channel_base
state_base
```

```
activation_destination
kernel_fold_id
```

建议采用每个 Macro 一块分布式、分 Bank 的 State SRAM，使 16 个 Cluster 能并行读取和写回各自负责的状态：

```
State SRAM in Macro m:
    bank 0 → lane 0 states, fold 0 ... F-1
    bank 1 → lane 1 states, fold 0 ... F-1
    bank 2 → lane 2 states, fold 0 ... F-1
    bank 3 → lane 3 states, fold 0 ... F-1
```

完整状态地址建议为：

```
state[layer_id][context_id][kernel_fold_id]
    [macro_id][mac_lane][spatial_position]
```

这里对“是否每个 IMC Cluster 都必须配置膜电位缓存”的结论是：

必须具备：每个正在工作的 Cluster 都要有与 4 路 Neuron/LIF lane 匹配的状态读写端口，并至少保存当前正在计算的 4 个神经元状态或当前 spatial tile 工作集。
不必强制：每个 Cluster 都在本地保存全部 F 个 Fold、全部空间位置的完整膜电位。
可选实现：完整状态可放在每 Macro 的分布式 State SRAM，或放在共享后备 State SRAM；Cluster 本地仅配置当前 Fold/tile 的工作缓存。

三种时间调度模式对本地缓存的要求如下：

模式	状态访问特征	Cluster 侧最低配置	推荐配置
TM, $B = 1$	每个 timestep 都遍历层和全部 Fold，状态读写最频繁	4-lane 当前状态寄存器/小工作缓存	全部 Fold 状态放在每 Macro 分布式 SRAM，避免每步访问共享后备 SRAM
HYB, $1 < B < T$	当前 Fold 状态可连续复用 B 个 timestep	当前 Fold/spatial tile 工作缓存	工作缓存 ping-pong；容量允许时全部 Fold 本地驻留
LM, $B = T$	当前 Fold 状态可连续复用完整窗口	当前 Fold/spatial tile 工作缓存	可接受共享后备 SRAM，每个 Fold 通常每窗口换入、写回各一次

因此，三种模式都需要 Cluster 在 LIF 更新点能够低延迟访问膜电位，但“每个 Cluster 都配全部状态容量”是性能优化方案，不是功能正确性的绝对条件。对于当前 16-Macro 架构，若面积允许，优先采用每 Macro 4-bank 分布式 State SRAM；它尤其能缓解 TM 模式和 $F = n + 1$ 较大时的共享状态带宽压力。

物理实现可选择以下两种方案。

方案 A：全部 Fold 状态驻留在各 Cluster

每个 Cluster 的膜电位容量为：

$$M_{U,cluster} = 4FH_{out}W_{out}b_U$$

若同时保存突触电流、不应期和 ALIF 状态，则：

$$M_{state,cluster} = 4FH_{out}W_{out}(b_U + b_I + b_R + b_A)$$

优点：

TM、HYB、LM 均不需要频繁换入换出 Fold 状态
16 个 Macro 天然提供分布式状态带宽
Fold 切换只改变本地地址，不产生跨 Macro 状态搬运
模式切换和 checkpoint 的状态归属清晰

这是状态容量允许时的推荐方案。

方案 B：当前 Fold 工作缓存加后备 State SRAM

Cluster 内只保存当前 fold、当前 spatial tile 的工作状态：

$$M_{work,cluster} = 4P_{tile}(b_U + b_I + b_R + b_A)$$

所有 fold 的完整状态放在后备 State SRAM：

加载当前 fold/tile 状态
连续计算 1、 ΔT 或 T 个 timestep
写回当前 fold/tile 状态
切换到下一 fold/tile

本地工作缓存推荐使用 ping-pong，以重叠当前 fold 计算与下一 fold 状态预取。该方案面积更小，但 TM 下状态搬运最频繁，HYB 和 LM 的复用效率更高。

5.3 三种模式下的 Fold 和膜电位调度

以下使用 ΔT 表示 temporal block 中的时间步数，避免与 block 数量混淆。

TM: $\Delta T = 1$

```
for t in timesteps:
    for layer in layers:
        for fold in range(layer_desc[layer].kernel_fold_count):
            load_fold_state_if_needed(layer, fold)
            run_16_clusters_x_4_macs(layer, fold, t)
```

```
update_64_lif_states(layer, fold, t)
store_fold_state_if_needed(layer, fold)
```

若 Cluster 只缓存当前 fold，则每个 fold 在一个窗口内状态加载/写回 T 次；若全部 F 个 Fold 的状态均驻留在 Cluster 分布式 State SRAM，则只需切换本地地址。

HYB: $1 < \Delta T < T$

```
for block in temporal_blocks:
    for layer in layers:
        for fold in range(layer_desc[layer].kernel_fold_count):
            load_fold_state_once(layer, fold)
            for t in block:
                run_16_clusters_x_4_macs(layer, fold, t)
                update_64_lif_states(layer, fold, t)
            store_fold_state_once(layer, fold)
```

每个 Fold 的状态在本地连续更新 ΔT 次，后备状态流量相对 TM 约降低到：

$$A_{HYB} \approx 2 \left\lceil \frac{T}{\Delta T} \right\rceil N_{U,total}$$

LM: $\Delta T = T$

```
for layer in layers:
    for fold in range(layer_desc[layer].kernel_fold_count):
        load_fold_state_once(layer, fold)
        for t in all_timesteps:
            run_16_clusters_x_4_macs(layer, fold, t)
            update_64_lif_states(layer, fold, t)
        store_fold_state_once(layer, fold)
```

每个 Fold 在一个窗口内只加载和写回一次状态：

$$A_{LM} \approx 2N_{U,total}$$

三种模式可以共用同一套 16 个分布式 State SRAM 和 64 路物理 Neuron lane，只改变状态驻留时长和 Fold 切换节拍：

模式	当前 Fold 状态连续更新	每个 Fold 每窗口换入次数	层间 spike 缓冲
TM	1 timestep	T	最小
HYB	ΔT timestep	$\lceil T/\Delta T \rceil$	一个 temporal block
LM	T timestep	1	完整窗口

5.4 Partial Sum 折叠与状态更新边界

当前各个输出 kernel Fold 是不同输出卷积核，可分别更新各自的膜电位。如果同一输出通道还在输入通道、卷积核元素或权重位上继续折叠：

$$I[c, y, x, t] = \sum_{p=0}^{P-1} I^{(p)}[c, y, x, t]$$

则前 $P - 1$ 个子 fold 只能更新 PSUM，最后一个子 fold 完成后才允许：

$$U[c, y, x, t] = \lambda U[c, y, x, t - 1] + I[c, y, x, t] - V_{reset} S[c, y, x, t - 1]$$

不能对每个 partial sum 执行：

$$U \leftarrow \lambda U + I^{(p)}$$

否则泄漏项 λ 会被重复施加。任务描述符应加入：

```
kernel_fold_id
output_channel_base
partial_fold_id
partial_fold_count
psum_first
psum_last
state_update_enable
```

只有 **psum_last=1** 且 **state_update_enable=1** 时，Neuron Engine 才读取、更新并写回膜电位，同时产生输出 spike。

5.5 状态不能按 block 复制为独立样本

连续 EEG/LFP 流中的相邻 block 属于同一条时间序列，因此：

```
B1 的层 l 结束状态是 B2 的层 l 初始状态
B2 的层 l 结束状态是 B3 的层 l 初始状态
```

正常连续检测时：

$$U_{l,start}^k = \gamma_{normal} U_{l,end}^{k-1}$$

当模型定义要求严格连续时：

```
 $\gamma_{normal} = 1$ 
```

当系统希望在窗口边界抑制长期漂移时，可配置：

```
 $\gamma_{normal} = 0.6-0.9$ 
```

但该衰减属于模型语义，必须在三种模式中按相同逻辑时间边界执行，不能因为物理 block 大小不同而额外衰减，否则三模式将不再数值等价。

5.6 每层独立状态提交

流水中可能同时存在：

```
Stage1 处理 Bk+2
Stage2 处理 Bk+1
Stage3 处理 Bk
```

因此每层维护独立状态版本：

```
state_version[l]
last_committed_block[l]
last_committed_t[l]
```

块 (l, k) 只有在以下条件成立时才能启动：

$$Ready(l, k) = InputReady(l, k) \wedge StateReady(l, k) \wedge ResourceReady(l)$$

其中：

```
InputReady(l, k): l=0, 或前一层的同一 block 已生成
StateReady(l, k): k=0, 或本层前一 block 已提交状态
ResourceReady(l): MRAM cluster、buffer、ADC 和输出端口可用
```

5.7 双状态银行与检查点

为支持精确模式切换和刺激后恢复，推荐每层设置：

Active State Bank	当前推理持续更新
Checkpoint State Bank	保存最近一个全局一致时间前沿

一致检查点不能在最后一层到达 t_c 时才临时读取所有层的 Active State，因为前级层可能早已推进到 t_c 之后。正确做法是预先选择候选检查点边界 t_c ，每层在首次完成该边界时立即把当时状态写入自己的 Checkpoint State Bank：

```
Layer1 到达 t_c → snapshot State1(t_c)
Layer2 到达 t_c → snapshot State2(t_c)
...
LayerL 到达 t_c → snapshot StateL(t_c)
```

只有当所有层的快照、readout 状态、风险 EMA、确认计数器和输出提交前沿均对应同一 t_c 时，才置位：

```
checkpoint_valid = 1
checkpoint_time = t_c
checkpoint_sequence = sequence_id(t_c)
```

这允许系统丢弃检查点之后的在途任务，并从 Input Ring 回放，而不会把 Stage1 的未来状态与 Stage3 的过去状态错误拼接。单个 Checkpoint State Bank 尚未完成全层快照时不得被下一候选边界覆盖；可通过增大 checkpoint 间隔、暂缓前级 snapshot，或增加第二个 checkpoint bank 解决检查点候选重叠。

6. 缓冲与互连设计

6.1 缓冲层次

L0: cluster 内输入寄存器、ADC 输出寄存器、partial-sum 寄存器
 L1: 每个 stage 的小型 ingress/egress FIFO
 L2: 共享 Activation SRAM, 支持 block 和 full-window ping-pong
 L3: Input Replay Ring, 保存模式切换和异常恢复所需原始 spike

6.2 三种模式的缓冲使用

模式	层间数据粒度	主要缓冲	缓冲压力
TM	单 timestep	小 FIFO / 寄存器	最低
HYB	B 个 timestep	block ping-pong + FIFO	中等
LM	T 个 timestep	full-window ping-pong SRAM	最高

为使同一硬件完整支持 LM，物理容量必须按最大窗口设计。若 spike 采用 bit-packed 存储，两个相邻层的 ping-pong 容量近似为：

$$M_{act} \geq T_{max} \cdot \max_l (N_l + N_{l+1}) \cdot b_s$$

其中二值 spike：

$$b_s = 1$$

若使用稀疏事件列表，则实际容量取决于最坏 spike rate。闭环安全系统不应只按平均稀疏率配置容量，应提供：

稠密 bit-vector fallback
 稀疏列表溢出检测
 溢出时阻塞或切换编码格式

6.3 输入回放环形缓冲

Input Ring 至少覆盖：

$$T_{replay} \geq T_{checkpoint_interval} + T_{max_inflight} + T_{switch_margin}$$

每项包含：

```
timestamp
sequence_id
epoch_id
input_spike
artifact_flag
CRC/parity
```

6.4 可重构路由

Router 至少支持：

```
前一 stage FIFO → 下一 stage
Activation SRAM → 任意 layer cluster
cluster 输出 → FIFO 或 Activation SRAM
Input Ring → Layer1
Readout 输出 → Reorder Buffer
失效 stage 旁路或备用 tile 路由
```

TM 侧重低延迟 cut-through；HYB 侧重 block packet；LM 侧重 SRAM burst。三者使用同一物理 NoC，但采用不同仲裁权重和 burst 长度。

7. 统一任务描述与调度器

7.1 任务描述符

每个调度任务表示为：

```
Task = {
    epoch_id,
    window_id,
    block_id,
    layer_id,
    kernel_fold_id,
    output_channel_base,
    partial_fold_id,
    partial_fold_count,
    psum_first,
    psum_last,
    state_update_enable,
```

```

    timestep_begin,
    valid_timesteps,
    input_buffer_addr,
    output_buffer_addr,
    state_version,
    mode,
    priority,
    artifact_flag
}

```

7.2 Packet Metadata

层间 packet 保存：

```

epoch_id
sequence_begin / sequence_end
window_id / block_id
valid_timesteps
producer_layer
kernel_fold_id / output_channel_base
partial_fold_id / psum_last
state_version
artifact_flag
stim_context
CRC/parity
valid

```

epoch_id 用于 logical flush; **sequence_id** 用于输出排序和去重; **state_version** 防止旧任务错误写回新状态。

7.3 调度记分牌

记分牌记录：

```

input_ready[layer][block]
state_ready[layer][block][kernel_fold]
fold_ready[layer][block][kernel_fold]
psum_ready[layer][block][kernel_fold]
fold_complete_mask[layer][block]
layer_block_complete[layer][block]
buffer_credit[port]
cluster_busy[layer]
output_credit
epoch_valid

```

任务发射条件：

```

can_issue = (
    input_ready[layer][block]
    and state_ready[layer][block][kernel_fold]
    and fold_ready[layer][block][kernel_fold]
    and cluster_available(layer)
    and output_buffer_has_credit(layer)
    and task.epoch_id == current_epoch
)

```

7.4 三种调度策略

TM:

BLOCK_LEN = 1
 优先级: 最老 timestamp 优先
 路由: stage cut-through
 输出: 允许 timestep 级 readout

HYB:

1 < BLOCK_LEN < WINDOW_LEN
 优先级: 最老 ready block 优先
 同一 stage 对 block 内 timestep 连续执行
 不同 block 允许在不同 stage 重叠
 输出: block 级或 timestep 级带时间戳输出

LM:

BLOCK_LEN = WINDOW_LEN
 启用 LAYER_BARRIER
 一层完成全窗口后才释放下一层
 Activation SRAM 使用 full-window ping-pong
 输出: 窗口级, 或在最后一层计算时按时间戳突发输出

7.5 统一调度伪代码

```

def configure_mode(mode, window_T, requested_B):
    if mode == "TM":
        B = 1
        layer_barrier = False
    elif mode == "HYB":
        B = clamp(requested_B, 2, window_T - 1)
        layer_barrier = False

```

```

elif mode == "LM":
    B = window_T
    layer_barrier = True
else:
    raise ConfigurationError

return B, layer_barrier

def build_tasks(window, B, epoch):
    blocks = partition(window, B, allow_short_last_block=True)
    for k, block in enumerate(blocks):
        for l in range(num_layers):
            for f in range(layer_desc[l].kernel_fold_count):
                scoreboard.register(
                    Task(epoch_id=epoch,
                        window_id=window.id,
                        block_id=k,
                        layer_id=l,
                        kernel_fold_id=f,
                        output_channel_base=(
                            layer_desc[l].parallel_output_channels * f
                        ),
                        partial_fold_id=0,
                        partial_fold_count=1,
                        psum_first=True,
                        psum_last=True,
                        state_update_enable=True,
                        timestep_begin=block.begin,
                        valid_timesteps=block.length)
                )

def scheduler_tick():
    ready = scoreboard.get_ready_tasks(current_epoch)
    task = policy.select_oldest_critical_task(ready)

    if task is None:
        return

    result, next_state = clusters[task.layer_id].run_block(task)

    if task.epoch_id != current_epoch:
        discard(result, next_state)
        return

    commit_layer_state(task, next_state)
    publish_output(task, result)
    scoreboard.complete(task)

    scoreboard.fold_complete_mask[task.layer_id][task.block_id].set(
        task.kernel_fold_id
    )
    if scoreboard.all_kernel_folds_complete(task.layer_id, task.block_id):

```

```
scoreboard.release_logical_layer_output(task.layer_id,
task.block_id)
```

`publish_output()` 在中间卷积层表示将该 fold 的输出通道写入其专属 Activation SRAM 地址，并不代表整个逻辑层已经完成。只有 `fold_complete_mask` 覆盖该层全部 F_l 个 kernel fold 后，才置位 `layer_block_complete`，并允许下一层消费完整的 $C_{out,l}$ 个逻辑输出通道；整组映射时为 $64F_l$ 个通道。

7.6 尾块处理

当 T 不能整除 B 时：

```
最后一个 block 的 valid_timesteps < BLOCK_LEN
无效 lane 不访问 MRAM、不更新神经元状态、不进入风险融合
packet 中显式携带 valid_timesteps
禁止通过补零产生额外的时间衰减
```

8. 输出语义与风险融合

8.1 物理完成时间与逻辑时间分离

每个结果同时携带：

```
logical_timestamp: 该结果对应的输入时间
completion_time: 硬件实际完成时间
```

TM 中两者接近；HYB 中结果按 block 延迟到达；LM 中一个窗口的早期时间戳结果可能在窗口后突发到达。风险控制只能按 `logical_timestamp` 顺序更新，不能按任意 cluster 的物理返回顺序更新。

8.2 输出模式

Readout 支持：

```
TIMESTEP_OUTPUT: 每个有效 timestep 产生 logit/risk
BLOCK_OUTPUT: 对一个 block 聚合后输出
WINDOW_OUTPUT: 对完整 detection window 聚合后输出
```

为了比较三种执行模式的数值一致性，推荐验证时使用 `TIMESTEP_OUTPUT`；为了降低闭环输出带宽，实际 HYB 可使用 `BLOCK_OUTPUT`，LM 可使用 `WINDOW_OUTPUT`。

8.3 风险平滑

每个有效输出：

$$r_{ema}(n) = \lambda r_{ema}(n-1) + (1-\lambda)r_{raw}(n)$$

推荐：

$$\lambda = 0.85-0.95$$

必须明确 n 的语义：

若三模式希望风险算法完全一致， n 应对应固定逻辑时间间隔
若 n 对应 block，则改变 B 时必须换算 λ ，保持相同物理时间常数

若参考时间步平滑系数为 λ_1 ，块长为 B ，等效块级系数可取：

$$\lambda_B = \lambda_1^B$$

8.4 Priority Fusion

窗口包含多个 block 时：

$$Y = \sum_{k=1}^K \alpha_k Y_k$$

$$\sum_{k=1}^K \alpha_k = 1$$

可定义：

$$q_k = f(r_k, c_k, e_k, age_k) - \eta_a artifact_k - \eta_s poststim_k$$

$$\alpha_k = \text{softmax}(q_k)$$

其中：

r_k : 局部风险
 c_k : 分类置信度或 margin
 e_k : spike energy / membrane evidence
 age_k : 逻辑时间新鲜度
 $artifact_k$: 刺激伪迹标记
 $poststim_k$: 刺激后上下文标记

8.5 闭环触发

```
if risk_ema > threshold_high:
    confirm_counter += 1
else:
    confirm_counter = 0
```

```
if confirm_counter >= confirm_required and cooldown_timer == 0:
    request_stimulation()
```

若改变输出粒度，`confirm_required` 也应按物理时间换算，不能把“连续 2 个 timestep”和“连续 2 个 8-timestep block”视为同一确认时长。

9. 运行时模式选择

9.1 软件显式选择

主控可以直接写入：

```
MODE = TM / HYB / LM
BLOCK_LEN = B
WINDOW_LEN = T
SWITCH_POLICY = DRAIN / CHECKPOINT_REPLAY / FORCE_RESET
```

9.2 自适应选择

推荐闭环场景采用：

低风险、稳定基线：LM 或大 B 的 HYB，降低调度和外围开销
 风险开始上升：中等 B 的 HYB，提高输出频率
 接近刺激阈值：TM，获得最低确认和触发延迟
 刺激期间：冻结、衰减或专用 artifact 模式
 刺激后 cooldown：小 B 的 HYB 或 TM，密切监测复发

示例策略：

```
if stim_active:
    target_mode = "TM"          # 仅做伪迹监测/状态衰减
elif risk_ema >= risk_critical:
    target_mode = "TM"
elif risk_ema >= risk_watch:
    target_mode = "HYB"
    target_B = 2
elif input_rate_high:
    target_mode = "HYB"
    target_B = 4
else:
    target_mode = "HYB"          # 闭环常开系统通常不建议长期使用大窗口 LM
    target_B = 8
```

LM 更适合作为：

离线数据回放
历史窗口复核
后台批处理
低风险且允许长反馈周期的监测
吞吐/能耗基准模式

对于癫痫刺激主通路，是否允许进入 LM 必须由系统最大可接受触发延迟约束决定。

9.3 基于延迟约束选择块长

设允许的最大算法分块等待为 D_{budget} ，单个 SNN 时间步代表 Δt_{snn} ，则：

$$B \leq \left\lfloor \frac{D_{budget} - D_{compute} - D_{control}}{\Delta t_{snn}} \right\rfloor$$

最终块长还需满足缓冲和稳定吞吐约束：

$$B^* = \max\{B : D(B) \leq D_{budget}, M(B) \leq M_{available}, II(B) \leq B\Delta t_{snn}\}$$

10. 安全模式切换协议

模式切换不能只修改 **MODE** 和 **BLOCK_LEN**。当 pipeline 中存在不同时间前沿的 block 时，直接切换会导致：

旧模式 packet 被新模式重复计算
Layer1 已推进到未来，Layer3 仍停留在过去
同一 sequence_id 输出两次
旧 epoch 高风险结果在切换后触发刺激

因此提供三种切换策略。

10.1 DRAIN：无回放精确切换

适合普通风险水平和非紧急切换：

1. 接收 mode_request，锁存 pending configuration
2. 停止 Block Builder 向调度器发射新 block，但 Input Ring 继续采集输入
3. 允许当前 epoch 的在途任务继续完成
4. 等待 FIFO 空、cluster idle、reorder buffer 提交完毕
5. 在当前逻辑时间边界建立状态检查点
6. 原子更新 MODE、BLOCK_LEN、路由和 buffer partition
7. 清零新模式局部计数器
8. 恢复 Block Builder 发射，并处理 Input Ring 中切换期间积累的输入

优点：

不丢输入
不重复计算
无需状态回滚
实现最简单

代价：

切换延迟等于当前在途数据排空时间

若预计排空期间 Input Ring 空间不足，则不启动 DRAIN；控制器改用 CHECKPOINT_REPLAY、临时背压上游，或进入预先定义的安全降级策略。闭环系统不得静默覆盖尚未处理的输入。

10.2 CHECKPOINT_REPLAY：低延迟精确切换

适合风险快速上升，需要从 LM/HYB 尽快切入 TM：

1. 冻结新任务发射
2. 读取最近全局一致 `checkpoint_time=t_c`
3. `current_epoch += 1`，使旧在途 `packet` 逻辑失效
4. 丢弃未提交输出，阻止旧 `state_version` 写回
5. Active State Bank、readout、risk_ema 和 confirm_counter 恢复到 `t_c`
6. 从 Input Ring 回放 `t_c+1` 之后的输入
7. 按新模式重建 block 和任务
8. 在追赶到切换前已对外提交的 `sequence frontier` 之前，屏蔽重复输出和控制副作用
9. Reorder Buffer 使用 `sequence_id` 去重；追赶完成后恢复正常输出与刺激判断

优点：

无需等待深流水完全排空
可快速进入低延迟模式
保持逐时间戳精确语义

代价：

需要双状态银行和 Input Ring
需要保存 readout/risk/controller 的一致检查点
会重复执行 checkpoint 之后的一小段计算

如果刺激命令等不可撤销副作用已经发出，不使用 CHECKPOINT_REPLAY 回滚到该副作用之前，而应进入刺激状态机并采用 FORCE_RESET/POST_STIM 流程。

10.3 FORCE_RESET：紧急非连续切换

仅用于刺激结束、系统故障或明确允许丢弃历史状态时：

1. `current_epoch += 1`
2. 全部 FIFO 和任务 `logical flush`
3. 神经元状态 `reset` 或按 `$\gamma_{stim}$` 衰减
4. 风险状态降权
5. 直接提交新模式

该策略不保证与未切换执行数值等价，但能提供最短恢复时间。

10.4 原子配置提交

配置寄存器分为：

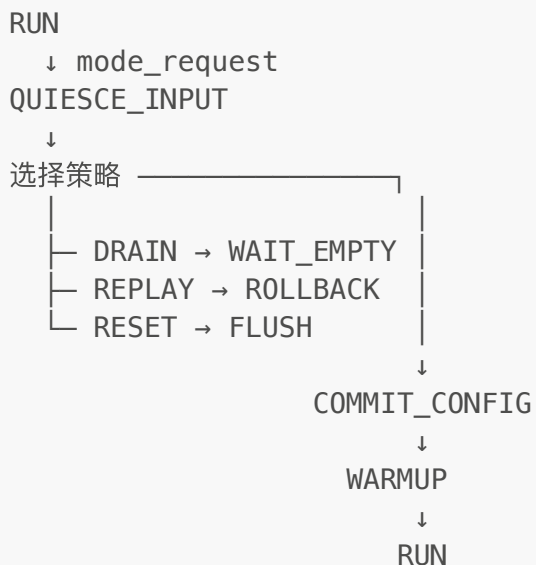
SHADOW：软件或自适应控制器写入
ACTIVE：硬件实际执行使用

只有在 `switch_safe=1` 时执行：

```
ACTIVE <= SHADOW
config_version <= config_version + 1
```

每个 task 携带 `config_version`，旧版本任务禁止写回新配置上下文。

10.5 模式切换状态机



WARMUP 用于重新填充 pipeline，并屏蔽不满足输出完整性的短暂结果。

11. 刺激后 Flush、状态处理与闭环状态机

11.1 刺激结束后的残留

系统可能残留：

神经元状态：U、I_{syn}、refractory、adaptive threshold
 pipeline 状态：FIFO、Activation SRAM、未完成 task
 风险状态：risk_ema、confirm counter、priority history
 刺激伪迹：输入 spike 突发和异常同步活动

11.2 Epoch Logical Flush

刺激结束后：

```
current_epoch += 1
```

所有旧 packet：

```
if packet.epoch_id != current_epoch:
    discard(packet)
```

物理 FIFO 可延迟清空，但旧数据不能进入状态写回、风险融合或刺激判断。

11.3 神经元状态软复位

$$U_l \leftarrow \gamma_{stim} U_l$$

$$I_{syn,l} \leftarrow \gamma_{stim} I_{syn,l}$$

```
refractory_l ← 0
adaptive threshold ← baseline 或按策略衰减
```

推荐：

```
γ_stim = 0.0-0.3
```

11.4 风险状态处理

```
risk_ema ← γ_risk · risk_ema
confirm_counter ← 0
```

priority_buffer 清空旧 epoch
reorder_buffer 丢弃旧 epoch

推荐：

$\gamma_{risk} = 0.2-0.5$

11.5 Artifact Blanking

推荐：

blanking_time = 1-2 个 HYB 基准 block 对应的物理时间

blanking 期间可配置：

输入强制为 0
只记录，不更新风险
只执行状态自然衰减
artifact_flag=1 并在 fusion 中 mask

Blanking 应按物理时间或逻辑 timestep 数配置，而不是简单按当前 block 数配置，否则切换模式后 blanking 时长会变化。

11.6 Cooldown

允许继续推理
禁止再次刺激
记录 residual seizure / suppression failure
高风险时可强制保持 TM
cooldown 到期后恢复正常触发资格

11.7 闭环状态机

```

NORMAL_DETECT
  ↓ 风险进入观察区
HIGH_RATE_MONITOR          可切 HYB(B=2) 或 TM
  ↓ 连续确认
PRE_STIM_CONFIRM
  ↓ confirmed
STIM_ACTIVE
  ↓ stimulation finished
POST_STIM_FLUSH_RESET      epoch++ / flush / state decay
  
```

```

↓
ARTIFACT_BLANKING
↓
COOLDOWN_MONITOR          建议 TM 或小 B HYB
↓ cooldown expired
NORMAL_DETECT

```

12. 性能与容量模型

12.1 单层处理时间

定义第 l 层第 f 个输出卷积核 fold 处理长度为 B 的 block 所需周期：

$$C_{l,f}(B) = C_{setup,l,f} + B \cdot C_{step,l,f} + C_{tail,l,f}$$

其中：

C_{setup} : fold 权重/状态基址选择、DMA、地址和 ADC 唤醒等固定开销
 C_{step} : 该 fold 每个 timestep 的阵列读取、ADC、累加和神经元更新
 C_{tail} : 该 fold 的输出提交、CRC、状态写回等开销

第 l 层具有 F_l 个输出卷积核 fold 时，完整逻辑层的 block 处理周期为：

$$C_l(B) = \sum_{f=0}^{F_l-1} C_{l,f}(B)$$

其中第 l 层的 Fold 数由逻辑输出通道数和 64 路物理并行宽度决定：

$$F_l = \left\lceil \frac{C_{out,l}}{64} \right\rceil$$

对于“首组 64 通道加额外 n_l 个完整折叠组”的层，有 $F_l = n_l + 1$ 且 $C_{out,l} = 64(n_l + 1)$ 。必须完成从 Fold 0 到 Fold $F_l - 1$ 的全部逻辑输出通道，才算该层当前 temporal block 完成。单个 fold 的平均每 timestep 周期为：

$$\bar{C}_{l,f}(B) = \frac{C_{setup,l,f} + C_{tail,l,f}}{B} + C_{step,l,f}$$

因此增大 B 能摊薄固定开销，但不会减少网络本身需要完成的有效 MAC/AC 数量。

12.2 空间 stage 流水性能

对于 $K = \lceil T/B \rceil$ 个 block，忽略尾块不平衡时：

首个 block 端到端计算延迟：

$$D_{first}(B) = \sum_{l=1}^L C_l(B) + C_{readout}$$

稳定流水启动间隔：

$$II(B) = \max_l C_l(B)$$

一个窗口总完成时间近似：

$$C_{window}(B) = \sum_{l=1}^L C_l(B) + (K-1)II(B) + C_{readout}$$

严格 LM 中 $K = 1$ ：

$$C_{window}^{LM} = \sum_l C_l(T) + C_{readout}$$

12.3 折叠单 cluster 性能

若所有层共享一个不可并行 cluster：

$$C_{window}^{folded}(B) = \sum_{k=1}^K \sum_{l=1}^L C_l(B_k)$$

此时 HYB 的主要收益来自减少配置与上下文切换，而不是层间并行。

12.4 闭环反馈延迟

从异常证据进入当前 block 到风险可用，近似为：

$$D_{feedback}(B) = D_{block_wait}(B) + D_{compute}(B) + D_{fusion} + D_{controller}$$

最坏分块等待：

$$D_{block_wait,max} \approx (B-1)\Delta t_{snn}$$

因此：

TM：分块等待近似为 0
HYB：最大等待随 B 线性增加
LM：最大等待接近完整窗口长度

12.5 实时稳定条件

每个长度为 B 的 block 每 $B\Delta t_{snn}$ 到达一次。稳定运行需要：

$$II(B) \leq B\Delta t_{snn} \cdot f_{clk}$$

若不满足：

增加并行 tile
提高 cluster 时钟
减小网络或位宽

采用更大 B 摊薄固定开销
允许输入排队并监测 backlog

注意：增大 B 会改善硬件服务效率，但也会增加闭环等待，不能只按吞吐选择。

12.6 存储容量

状态存储：

$$M_{state} = n_{bank} \sum_l N_l (b_U + b_I + b_R + b_A)$$

对于采用本方案 64 路物理并行和输出 kernel fold 的卷积层：

$$N_l = C_{out,l} H_{out,l} W_{out,l}$$

整组映射时 $C_{out,l} = 64F_l = 64(n_l + 1)$ 。因此，增加输出卷积核 Fold 不增加物理 MAC/LIF lane 数量，但会按 F_l 线性增加逻辑膜电位状态容量、输出激活容量和执行周期。

其中支持检查点时：

n_bank = 2

输入回放：

$$M_{replay} = T_{replay} \cdot N_0 \cdot b_s + M_{metadata}$$

激活存储按第 6.2 节的 full-window 最坏需求配置。

12.7 能耗模型

每窗口能耗可分为：

$$E(B) = E_{MRAM_read} + E_{ADC} + E_{neuron} + E_{buffer}(B) + E_{NoC}(B) + E_{control}(B)$$

其中：

有效阵列读取主要由网络与 spike activity 决定
块级控制、ADC 唤醒和地址开销通常随 block 数 K 增加
Activation SRAM 能耗通常随 B 增大，LM 最高
TM 的小 FIFO 能耗低，但控制和频繁提交开销高
HYB 的最优 B 需要实测，不应预设必然为 4

13. 配置寄存器建议

寄存器	字段	说明
MODE_CTRL	mode[1:0]	00=TM, 01=HYB, 10=LM

寄存器	字段	说明
MODE_CTRL	switch_policy[1:0]	drain、checkpoint replay、force reset
MODE_CTRL	request/ack/busy/error	模式切换握手
BLOCK_CFG	block_len	HYB 块长，TM 固定 1，LM 固定 T
WINDOW_CFG	window_len	检测窗口时间步数
SCHED_CFG	layer_barrier	LM 必须为 1
SCHED_CFG	oldest_first/qos	调度优先级
FOLD_CFG	kernel_fold_count	当前层输出卷积核 Fold 总数 F_l ，可按层编程
FOLD_CFG	extra_fold_count	首组之外的额外完整折叠组数 $N_{add,l} = n_l$ ，整组映射时 $F_l = n_l + 1$
FOLD_CFG	parallel_channels	物理并行输出通道数，当前为 64
FOLD_CFG	valid_channels_last_fold	最后一个 Fold 的有效输出通道数，完整 64 通道时写 64
FOLD_CFG	partial_fold_count	同一输出的输入/位切片 PSUM fold 数
FOLD_CFG	fold_weight_base[]	各输出卷积核 fold 的权重地址
FOLD_CFG	fold_state_base[]	各输出卷积核 fold 的状态地址
STATE_CFG	gamma_normal	正常状态边界衰减
STATE_CFG	gamma_stim	刺激后状态衰减
STATE_CFG	checkpoint_interval	一致检查点间隔
OUTPUT_CFG	granularity	timestep、block、window
OUTPUT_CFG	reorder_enable	按 sequence_id 提交
RISK_CFG	lambda/thresholds	EMA 和双阈值
STIM_CFG	confirm_time	建议按逻辑时间配置
STIM_CFG	blanking_time	刺激伪迹屏蔽时长
STIM_CFG	cooldown_time	冷却时长
EPOCH	current_epoch	logical flush 标识
STATUS	inflight/fifo_level	在途任务和缓冲占用
STATUS	committed_timestamp	全局提交时间前沿
ERROR	overflow/crc/ecc/timeout	安全错误状态

配置合法性检查：

TM: block_len 必须为 1
 HYB: $1 < \text{block_len} < \text{window_len}$
 LM: block_len 必须等于 window_len, layer_barrier 必须为 1
 block_len 不得超过 Activation SRAM 可用容量
 checkpoint replay 必须启用 Input Ring 和双状态银行
 $\text{kernel_fold_count} = \text{ceil}(\text{logical_output_channels} / \text{parallel_channels})$
 $\text{kernel_fold_count} \times \text{parallel_channels}$ 必须覆盖逻辑输出通道数
 valid_channels_last_fold 必须与逻辑输出通道余数一致, 余数为 0 时配置为 64
 不同 kernel fold 的 state address range 不得重叠
 partial_fold_count > 1 时, 只有最后一个 partial fold 可置 state_update_enable

非法配置不进入 ACTIVE, 并置位 `config_error`。

14. 推荐第一版硬件配置

14.1 网络与时间参数

输入: EEG/LFP feature spike stream
 目标卷积层物理并行通道 $C_{\text{parallel}} = 64$
 首组之外的额外折叠组数 $N_{\text{add}} = n$, 按层可配置
 输出卷积核 Fold 总数 $F = n + 1$
 完整 64 通道组映射时逻辑输出通道 $C_{\text{out}} = 64(n + 1)$
 非完整末组时 $F = \text{ceil}(C_{\text{out}} / 64)$
 输出特征图尺寸 $H_{\text{out}} \times W_{\text{out}}$ 由具体网络层 descriptor 配置
 后续分类/readout 层输出维度可配置为 2 (正常/高风险)
 window_len $T = 16$ timestep
 TM block_len = 1
 HYB block_len = 4, 可配置 2/4/8
 LM block_len = 16

14.2 MRAM 与数字外围

Macro 数量 = 16
 每个 Macro 包含 1 个 Cluster
 每个 Cluster 包含 4 个卷积 MAC lane
 物理并行输出通道数 $= 16 \times 4 = 64$
 额外折叠组数 $\text{extra_fold_count} = n$
 输出卷积核 $\text{kernel_fold_count} = F = n + 1$
 整组映射时逻辑输出通道数 $= 64(n + 1)$
 Fold f 映射 output channel $64f \dots 64f+63$, $f = 0 \dots F-1$
 末 Fold 不满 64 通道时由 valid_channels_last_fold 屏蔽无效 lane
 每个 Cluster 配置 4 路并行 Neuron/LIF lane
 每个 Macro 配置 4-bank 分布式 State SRAM
 状态组织 = $U[\text{fold}][\text{mac_lane}][\text{spatial_position}]$
 推荐全部 F 个 Fold 状态在所属 Macro 内驻留; 容量不足时采用当前 Fold 工作缓存加后备 State SRAM
 weight_bits = 6

```
cell_effective_bits = 2
ADC_bits = 6
digital accumulator = 16 bit
membrane/state = 16 bit fixed point
signed weight = differential mapping
FIFO depth per link = 4 block packets
Activation SRAM = 至少支持 T=16 的双缓冲
State SRAM = active + checkpoint 双银行, 均包含 fold 维度
Input Ring = 至少 2 个窗口
```

14.3 状态与闭环参数

```
 $\gamma_{\text{normal}} = 1.0$  (先验证严格连续性)
可选窗口边界  $\gamma_{\text{normal}} = 0.8$ 
 $\gamma_{\text{stim}} = 0.2$ 
risk  $\lambda_{\text{timestep}} = 0.9$ 
threshold_high = 0.7
threshold_low = 0.4
confirm duration = 2–4 个 timestep 对应物理时间
blanking duration = 4–8 个 timestep 对应物理时间
cooldown duration = 16–32 个 timestep 对应物理时间
```

14.4 初始运行策略

默认: HYB, B=4
risk_ema ≥ 0.55 : 切 HYB, B=2
risk_ema ≥ 0.65 : CHECKPOINT_REPLAY 切 TM
刺激后: FORCE_RESET + TM
cooldown 后: DRAIN 切回 HYB, B=4
LM: 先用于离线回放和性能基准, 不默认进入实时刺激主路径

阈值仅作为架构验证起点, 最终必须由数据集、患者特异模型和临床安全约束确定。

15. 编译、训练与软件栈

15.1 离线映射流程

```
训练后 SNN 模型
↓
依赖图分析与模式兼容性检查
↓
量化感知训练 / 非理想性注入
↓
权重差分编码与 bit slicing
↓
MRAM tile placement
```

↓
生成 Layer Descriptor Table
↓
生成三模式调度参数和 buffer layout
↓
下载权重与配置

15.2 模式兼容性报告

编译器应输出：

```
supported_modes
maximum_legal_block_len
required_sync_barriers
activation_memory_per_mode
state_memory
estimated_latency/throughput/energy
unsupported_recurrent_edges
```

15.3 训练建议

QAT 中加入权重、ADC、累加器和膜电位量化
注入 MRAM 电导偏差和 ADC offset/gain 噪声
刺激伪迹样本参与训练或校准
风险 readout 使用固定逻辑时间定义
模式仅改变执行调度时，不必为每种模式训练三份权重
若不同模式采用不同输出聚合，则聚合头需联合训练或分别校准

16. 验证方案

16.1 算法级等价验证

对同一输入和同一初始状态，比较：

```
TM, B=1
HYB, B=2
HYB, B=4
HYB, B=8
LM, B=T
```

逐项检查：

每层每 timestep spike
 每层每 timestep membrane state
 每层最终 state
 逐 timestamp logit/risk_raw
 窗口最终输出

固定点模型应 bit-exact；含模拟噪声时比较统计误差分布。

针对可配置的输出卷积核折叠，额外检查：

任意 Fold f 只写 output channel $[64f, \min(64f+63, \text{Cout}-1)]$ 的激活和状态
 任意 $f1 \neq f2$ 时，同一 Macro/MAC lane 的 state address range 不重叠
 切换 Fold 后不残留上一 Fold 的 weight/state/output base
 全部 F 个 Fold 均完成后才置位 logical layer/block complete
 最后一个 Fold 不满 64 通道时，无效 lane 不读权重、不更新状态且不写输出
 输入/位切片 partial fold 未完成时不得更新膜电位
 每个逻辑 timestep 对每个输出神经元恰好执行一次 LIF 泄漏与阈值判断

16.2 调度依赖验证

断言：

不得在前一层输入未完成时启动任务
 不得在本层前一 block 状态未提交时启动任务
 旧 epoch 不得写回 state
 旧 config_version 不得提交输出
 同一 sequence_id 最多提交一次
 输出 sequence_id 必须单调递增

16.3 模式切换验证

覆盖：

TM → HYB → LM → TM
 任意 block 中间发起 DRAIN
 深流水中发起 CHECKPOINT_REPLAY
 状态写回周期发起 epoch flush
 FIFO 满、空、半满时切换
 尾块和窗口边界切换
 刺激同时到达 mode_request

检查：

不丢逻辑输入
不重复提交输出
状态回滚正确
回放范围正确
模式切换延迟可界定

16.4 MRAM-IMC 电路验证

评估：

PVT 下读电流分布
ADC ENOB、offset、gain 和饱和率
差分权重误差
位切片累加误差
IR drop 和阵列尺寸关系
读扰动与写耐久度
校准前后准确率

16.5 系统与闭环指标

算法指标：

sensitivity / specificity
AUROC / AUPRC
false trigger rate
missed seizure rate
post-stim repeated trigger

延迟指标：

evidence-to-risk latency
risk-to-stim latency
mode switch latency
checkpoint replay latency
pipeline first-output latency

硬件指标：

stage utilization
FIFO occupancy and stall rate
MRAM read count
ADC conversion count
SRAM/NoC traffic
energy per timestep

energy per risk decision
area and peak power

可靠性指标：

CRC/ECC correction count
stale epoch discard count
replay count
buffer overflow count
watchdog timeout count
unsafe configuration rejection count

16.6 推荐实验矩阵

实验组	模式/参数	目的
A	TM, $B = 1$	最低延迟基线
B	HYB, $B = 2$	小块折中
C	HYB, $B = 4$	推荐初版
D	HYB, $B = 8$	高复用折中
E	LM, $B = T$	吞吐/能耗基线
F	固定 HYB 与自适应 B	验证动态策略收益
G	DRAIN 与 CHECKPOINT_REPLAY	比较切换延迟及额外计算
H	无 flush、仅 flush、flush+reset+blanking	验证刺激后稳定性

17. 风险与设计取舍

17.1 全模式支持的主要面积代价

真正使 LM 可用的主要代价不是模式寄存器，而是：

full-window Activation SRAM
Input Replay Ring
双状态银行
更复杂的 Router 和 Reorder Buffer

如果面积紧张，可以降低 T_{max} 、压缩 spike、减少 checkpoint 深度，或把 LM 限制为较短窗口；但不能宣称支持任意长度 LM 而不配置相应激活存储。

17.2 模式切换并非零成本

DRAIN 有排空延迟
CHECKPOINT_REPLAY 有回放能耗和状态存储开销
FORCE_RESET 会改变模型时间连续性

因此自适应策略需要滞回和最短驻留时间，避免风险在阈值附近造成频繁模式振荡。

17.3 专用 stage 下的模式收益变化

若每层拥有独立 MRAM、ADC 和神经元单元，TM 本身也能形成高利用率波前流水，所谓“层切换开销”会明显减小。此时 HYB/LM 的收益更多来自：

块级控制摊销
SRAM burst 效率
ADC/NoC 功耗门控
输出聚合
任务描述符数量减少

若采用折叠共享外围，增大 B 对减少上下文切换的收益会更明显。性能报告必须注明物理映射，不能只根据循环顺序推断结论。

17.4 临床闭环安全

LM 的高反馈延迟必须经过上限验证
任何模式下旧 epoch 输出都不能触发刺激
模式控制器故障时应退回已验证安全模式，通常为 TM 或 HYB($B=1/2$)
刺激输出需要独立硬件看门狗、剂量上限和人工配置保护
本方案是工程架构设计，不能替代医疗器械安全、临床和法规验证

18. 分阶段实现路线

阶段 1：统一功能模型

实现参数化 B 的软件 golden model
证明 $B=1$ 、 $B=2/4/8$ 、 $B=T$ 的逐时间戳等价性
统一 EMA、确认时间和 blanking 的物理时间语义

阶段 2：数字 RTL 原型

先使用 SRAM 权重模型替代模拟 MRAM
实现 Layer Descriptor、Scoreboard、FIFO、State Bank 和 Reorder
实现 DRAIN 与 epoch logical flush

阶段 3：安全重构

加入 Input Ring、Checkpoint State 和 CHECKPOINT_REPLAY
验证所有模式切换与刺激竞态
加入 config shadow/active 原子提交

阶段 4：MRAM 宏模型接入

加入差分权重、ADC、位切片和非理想性模型
完成 QAT 与校准 LUT
统计阵列、ADC、SRAM 和 NoC 能耗

阶段 5：系统闭环验证

接入真实 EEG/LFP 数据流
加入刺激伪迹、丢包、缓冲拥塞和器件故障注入
比较固定模式与自适应模式
确定临床安全约束下允许的最大 B

19. 最终方案总结

本方案用一个可编程时间块长度 B 统一三种 SNN 执行方式：

$B=1$ ：逐时间步优先，最低闭环反馈延迟
 $1 < B < T$ ：时间块混合，块内层优先、块间时间流水
 $B=T$ ：全窗口层优先，最高连续复用和批处理效率

三种模式共享：

MRAM 常驻权重
IMC 计算簇
神经元状态存储
激活和输入缓冲
风险融合及刺激控制

真正实现“可重构”所需的关键并不是三选一多路器，而是以下五点：

1. **依赖正确的统一调度器**：同时保证同层时间状态依赖和跨层数据依赖；
2. **可覆盖全窗口的弹性缓冲**：TM 用小 FIFO，HYB 用 block buffer，LM 用 full-window ping-pong；
3. **版本化状态与 metadata**：使用 `epoch_id`、`sequence_id`、`state_version` 和 `config_version` 隔离旧任务；
4. **安全模式切换**：支持 DRAIN、CHECKPOINT_REPLAY 和 FORCE_RESET；
5. **按物理时间统一的闭环语义**：EMA、确认、blanking 和 cooldown 不随 block 粒度错误改变。

推荐第一版采用 `16 Macro × 1 Cluster/Macro × 4 Conv-MAC/Cluster` 的 64 通道物理并行结构。首组 64 个输出卷积核之外的额外 n 组卷积核不复制 Cluster，而是形成总计 $F = n + 1$ 个 kernel Fold，依次分时复用同一组 64 路卷积 MAC；完整组映射时可承载 $64(n + 1)$ 个逻辑输出通道。每个 Macro 配置 4-bank 分布式 State SRAM，使各 lane 在全部 F 个 Fold 上的膜电位状态彼此独立；若本地容量不足，则采用当前 Fold 工作缓存加后备 State SRAM。时间配置采用 $T = 16$ 、默认 HYB $B = 4$ ，并完整实现 TM、HYB、LM 三种模式。实时癫痫检测主路径默认使用 HYB，在风险上升时切换到 TM；LM 先用于离线回放、低风险批处理和性能基准。这样既保留附件中 temporal-block pipelined layer-major 的低延迟/高复用折中，也把两个极端调度及任意 n 组输出卷积核折叠纳入同一套可验证、可切换、可扩展的 MRAM 存内计算架构。